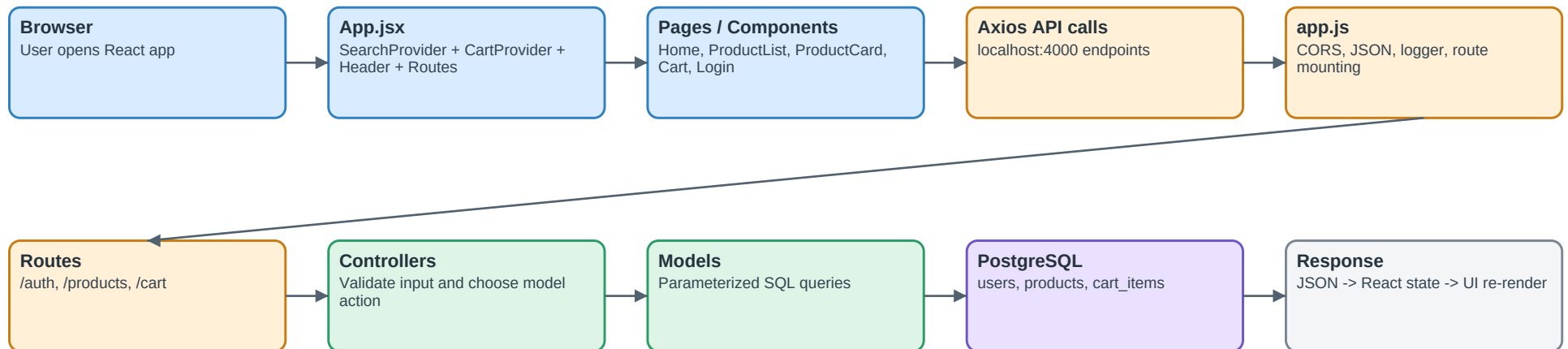


Complete Working Flowchart

React client -> Express routes -> controllers -> models -> PostgreSQL

This document maps the actual Minion project files. The frontend is a React/Vite app. The backend is an Express API using JWT auth, Joi validation, PostgreSQL models, and a central error handler.

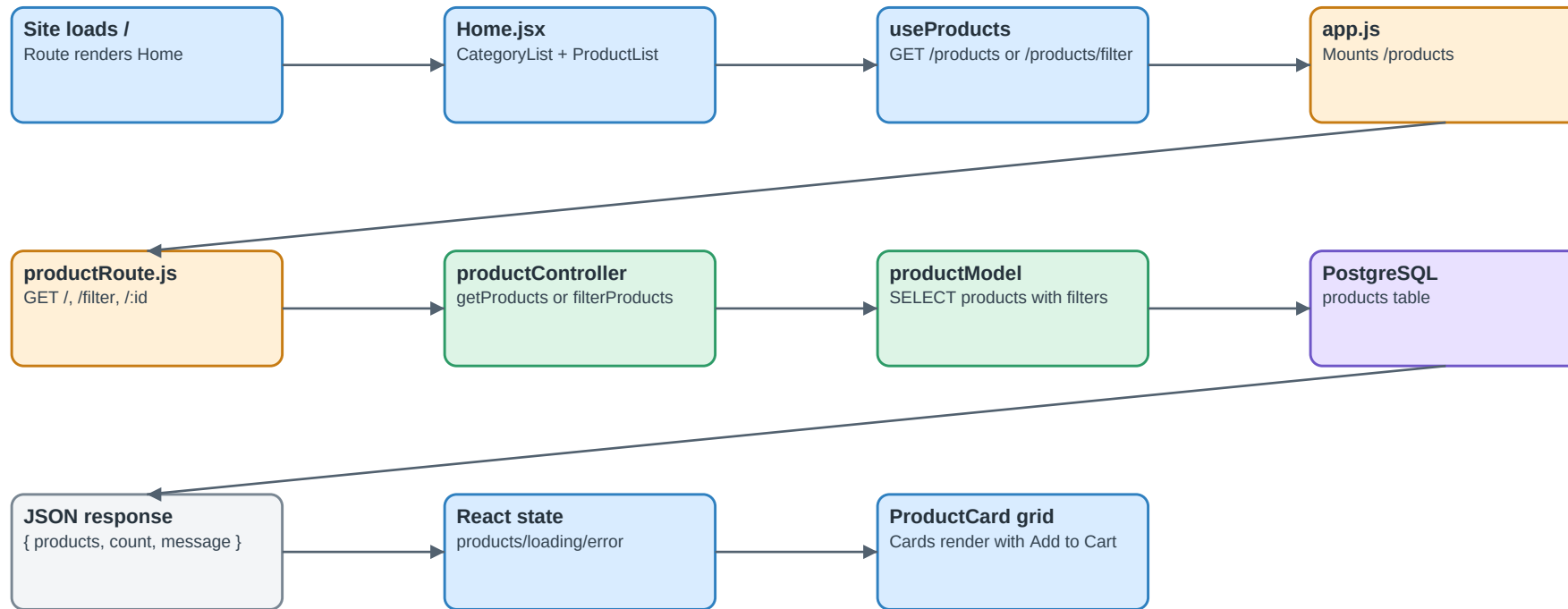


Key files

- Client: client/src/App.jsx, Routing/Route.jsx, Contexts/SearchContext.jsx, Contexts/CartContext.jsx, hooks/useProducts.js.
- Backend: backend/app.js, routes/*.js, controllers/*.js, models/*.js, middlewares/auth.js, schemas/*.js, db.js.

Loading Products, Search, and Categories

Public product read flow: no token required



Search flow

- Header form calls SearchContext.handleSubmit(searchText).
- SearchContext sets endpoint to /products/filter?name=...
- ProductList receives the new endpoint and useProducts refetches.
- Backend validates query with productSchema.filterSchema, then productModel.filter builds SQL WHERE clauses.

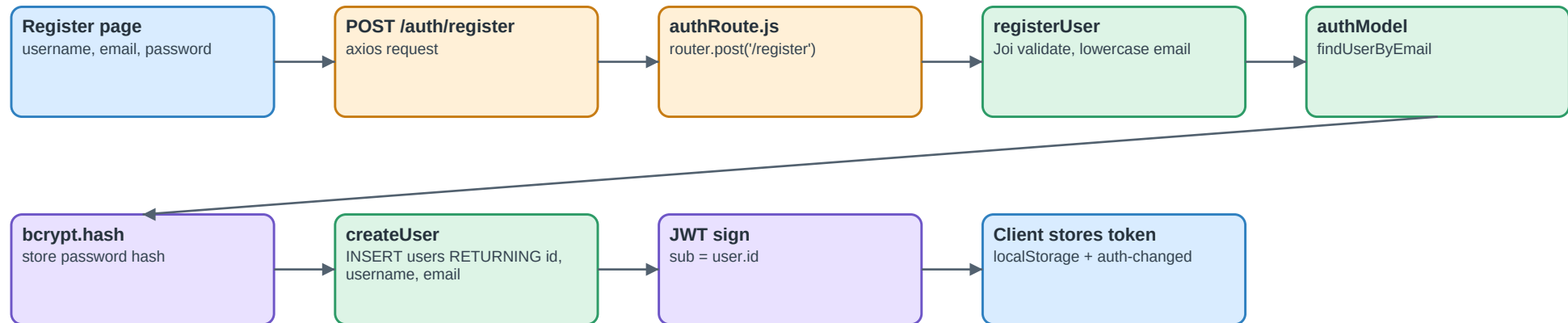
Category flow

- CategoryList separately loads /products to count categories.
- Clicking a category calls handleCategoryClick(category).
- The endpoint becomes /products/filter?category=...
- If no product matches, controller returns 404 with an empty products array and optional availableCategories.

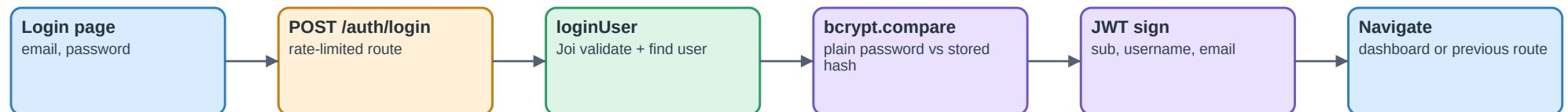
Authentication Workflow

Register/login create a JWT; protected APIs verify it before controller logic

Register



Login

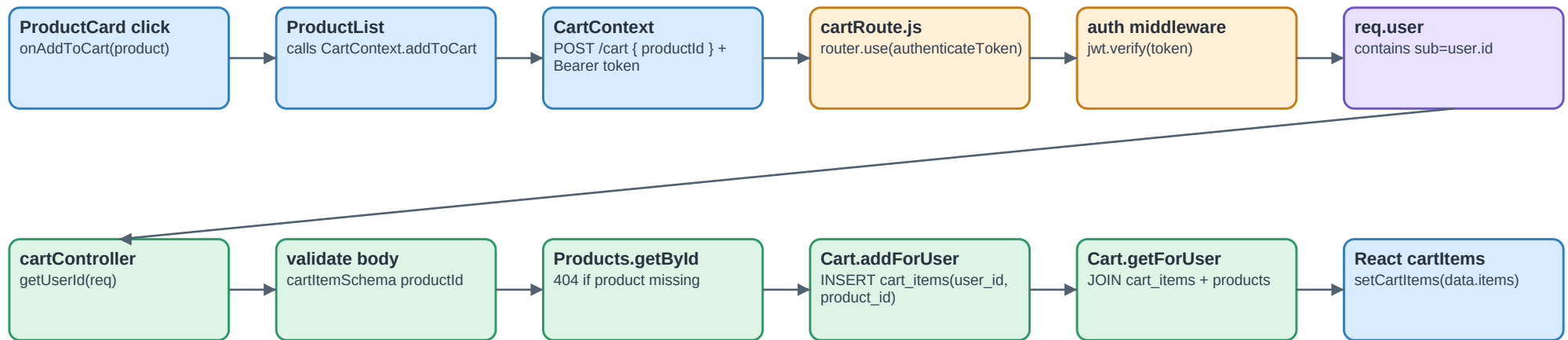


Important auth detail

- The token payload is created in authController.createToken with sub: String(user.id).
- Cart and protected product write routes do not search localStorage. They receive Authorization: Bearer <token>, verify it on the server, and read the user id from req.user.sub.

Cart Workflow and Auth User Lookup

Add to Cart, cart loading, and removal all depend on the verified JWT



When the app opens with a token

- CartProvider initializes token from localStorage.
- Its effect sends GET /cart with Authorization header.
- If the server returns 401, CartProvider removes the token, clears cartItems, and stops treating the user as logged in.

Remove from cart

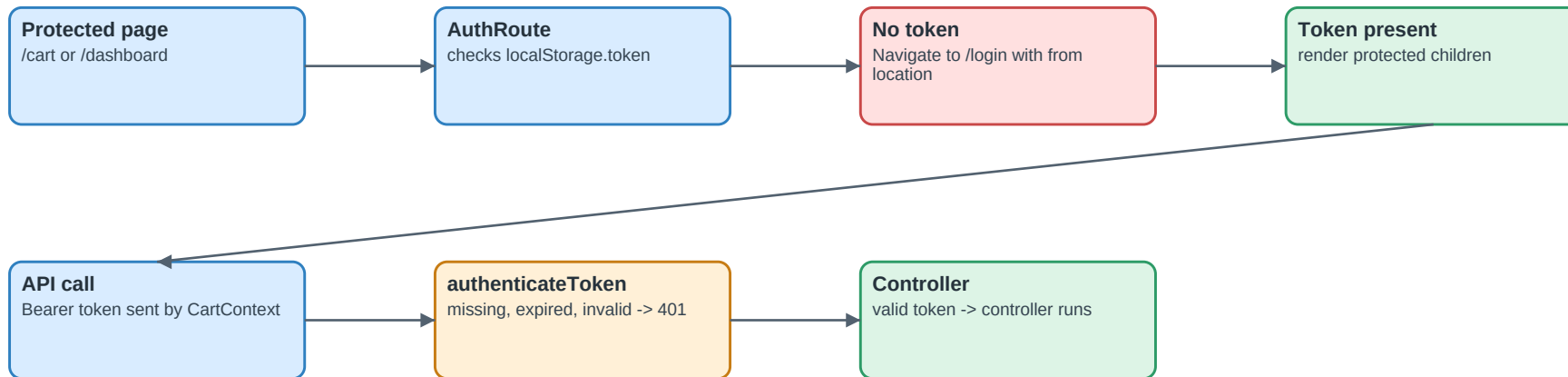
- Cart page renders protected by AuthRoute.
- Remove button calls removeFromCart(product.id).
- Client sends DELETE /cart/:productId with Bearer token.
- Controller validates productId, derives userId from req.user.sub, deletes that user's row, then returns the refreshed cart.

How the authenticated cart user is found

- Login/register returns a JWT whose sub claim is the database users.id. authenticateToken verifies the Bearer token and sets req.user to that decoded payload. cartController.getUserId converts req.user.sub to a number, checks it is valid, and passes that userId into Cart.getForUser, addForUser, and removeForUser.

Protected Routes, Validation, and Errors

What happens when a user is missing, input is invalid, or routes fail



Validation path

- Controllers call `validate(schema, data)`.
- Joi returns defaulted/validated values or validation details.
- `validate` throws `HttpError(422, joined Joi messages)`.
- `errorHandler` returns `{ success:false, message }`.

Product write path

- `POST/PUT/PATCH/DELETE /products` are protected.
- Route first runs `authenticateToken`.
- Controller validates request params/body.
- `productModel` executes `INSERT, UPDATE, DELETE, or TRUNCATE`.

Database protection

- `db.js` validates `DB_SCHEMA` before interpolating it.
- Values are passed as query parameters like `$1, $2`.
- `cart_items` has a primary key on `user_id + product_id`.
- `ON CONFLICT DO NOTHING` prevents duplicate cart rows.

End-to-end mental model

- Frontend state chooses endpoint and sends axios requests. Backend route decides middleware and controller. Controller validates and calls a model. Model talks to PostgreSQL. The JSON response updates React state, which re-renders the visible UI.